BHARATIYA VIDYA BHAVAN'S
SARDAR PATEL INSTITUTE OF TECHNOLOGY
Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai – 400058-India
Department of Computer Engineering

Name	Raj Kalpesh Mathuria
UID no.	2023300139
Experiment No.	4

AIM:	To calculate emission and transition matrix for tagging Parts of Speech using Hidden Markov Model. Find the POS tag of a given sentence using HMM.
PRE LAB QUESTIONS:	1. List a few ways for tagging parts of speech Parts of speech can be tagged using several approaches: Rule-based tagging – Uses manually written grammatical rules to assign tags based on word patterns and context. Statistical tagging – Uses probabilistic models like Hidden Markov Models (HMMs) that rely on transition and emission probabilities. Machine Learning methods – Use classifiers such as Naive Bayes, SVM, or Decision Trees trained on annotated corpora. Deep Learning approaches – Use models like RNNs, LSTMs, or Transformers that learn contextual patterns automatically from large datasets. 2. How do you find the most probable sequence of POS tags from a sequence of text? The most probable sequence of POS tags is found using the Viterbi Algorithm in a Hidden Markov Model. It computes the probability of all possible tag sequences using transition probabilities (between tags) and emission probabilities (word given tag). The algorithm efficiently selects the sequence with the highest overall probability instead of checking every possible combination. 3. Differentiate between Markov Chain and Markov Model A Markov Chain is a stochastic model that represents transitions between observable states where the next state depends only on the current state. A Markov Model is a broader concept that may include hidden states and emission probabilities, such as in a Hidden Markov Model (HMM). In short, a Markov Chain deals only with state transitions, while a Markov Model can include hidden layers and observed outputs. 4. How can you identify whether a system follows a Markov Process? A system follows a Markov Process if it satisfies the Markov Property, meaning the

	future state depends only on the present state and not on the sequence of past states. If knowing earlier history does not improve prediction beyond the current state, then the system can be modeled as a Markov Process. 5. Explain the use of Markov Chains in text generation algorithms Markov Chains are used in text generation by modeling the probability of a word occurring based on the previous word (or previous n words in higher-order models). By learning transition probabilities from a corpus such as the Brown Corpus, the model can generate new sentences that statistically resemble natural language. Though simple, this approach forms the foundation of many probabilistic language models.
THEORY:	**POS tagging or part-of-speech tagging** is the procedure of assigning a grammatical category like noun, verb, adjective etc. to a word. In this process both the lexical information and the context play an important role as the same lexical form can behave differently in a different context. For example the word "Park" can have two different lexical categories based on the context. The boy is playing in the park. ('Park' is Noun) Park the car. ('Park' is Verb). Assigning part of speech to words by hand is a common exercise one can find in an elementary grammar class. But here we wish to build an automated tool which can assign the appropriate part-of-speech tag to the words of a given sentence. One can think of creating handcrafted rules by observing patterns in the language, but this would limit the system's performance to the quality and number of patterns identified by the rule crafter. Thus, this approach is not practically adopted for building POS Taggers. Instead, a large corpus annotated with correct POS tags for each word is given to the computer and algorithms then learn the patterns automatically from the data and store them in form of a trained model. Later this model can be used to POS tag new sentences. **POS Tagging - Hidden Markov Model** A Hidden Markov Model (HMM) is a statistical Markov model in which the system being modeled is assumed to be a Markov process with unobserved (hidden) states. In a regular Markov model, the state is directly visible to the observer, and therefore the state transition probabilities are the only parameters. In a hidden Markov model, the state is not directly visible, but output, dependent on the state, is visible. Hidden Markov Model has two important components.

1)Transition Probabilities: The one-step transition probability is the probability of transitioning from one state to another in a single step.

2)Emission Probabilities: : The output probabilities for an observation from state. Emission probabilities B = { bi,k = bi(ok) = P(ok | qi) }, where ok is an Observation. Informally, B is the probability that the output is ok given that the current state is qi For POS tagging, it is assumed that POS are generated as a random process, and each process randomly generates a word.

PROBLEM SOLVING:

Name: Raj Mathuria
UID: 2023300139
Batch: PE - C

Page No.
Date

Corpus :- The /DET dog / NOUN barks/ VERB
A / DET cat / NOUN runs / VERB

Transition probability

START → First tag END → last tag

FROM SENTENCE 1 :- START → DET , DET → NOUN , NOUN → VERB , VERB → END

FROM SENTENCE 2 :- START → DET , DET → NOUN , NOUN → VERB , VERB → END

Transition Matrix

	D	N	V	E
START	1	0	0	0
DET	0	1	0	0
NOUN	0	0	1	0
VERB	0	0	0	1

From	To	Count
START	DET	2
DET	NOUN	2
NOUN	VERB	2
VERB	END	2

$P(Tag_2 | Tag_1) = \frac{\text{Count}(Tag_1 \to Tag_2)}{\text{Total transitions from } Tag_1}$

P(DET | START) = 1 = 2/2 = P(END | VERB)

P(NOUN | DET) = 1 = 2/2

P(VERB | NOUN) = 2/2 = 1

Emission probability

P(Word | Tag) → To find

DET → The , A NOUN → dog , cat VERB → barks , runs

Tag	Word	Count
DET	The	1
DET	A	1
NOUN	dog	1
NOUN	cat	1
VERB	barks	1
VERB	runs	1

$P(\text{word} | \text{tag}) = \frac{\text{Count}(\text{word}, \text{tag})}{\text{Total words with tag}}$

P(The | DET) = 1/2 = P(A | DET)

P(dog | NOUN) = 1/2 = P(cat | NOUN)

P(barks | VERB) = 1/2 = P(runs | VERB)

P(DET | START) = 1

P(END | VERB) = 1

Tagging using HMM

Sentence 1 :- DET → NOUN → VERB

Formula :- P = P(DET | START) × P(The | DET) × ..... × P(VERB | NOUN) × P(END | VERB)

= 1 × 1/2 × 1 × 1/2 × 1 × 1/2 × 1 = 1/8 = 0.125

ONLY ONE PATH → itself max

PROGRAM:

```
import numpy as np
import nltk
from nltk.corpus import brown
from collections import defaultdict, Counter
from itertools import product
from hmmlearn import hmm

nltk.download('brown')
nltk.download('universal_tagset')

np.set_printoptions(threshold=np.inf, linewidth=200)

tagged_sentence = [('Daniel', 'NOUN'), ('personally',
'ADV'), ('likes', 'VERB'), ('to', 'PRT'), ('read',
'VERB')]

print("\nSentence Used For Computation:\n")
print([word.lower() for word, tag in tagged_sentence])
print("\nTagged Version:\n")
print(tagged_sentence)

transition_counts = defaultdict(Counter)
emission_counts = defaultdict(Counter)
tag_counts = Counter()
start_counts = Counter()

if len(tagged_sentence) != 0:

   first_tag = tagged_sentence[0][1]
   start_counts[first_tag] += 1

   for i, (word, tag) in enumerate(tagged_sentence):
       word = word.lower()
       emission_counts[tag][word] += 1
       tag_counts[tag] += 1

       if i > 0:
           prev_tag = tagged_sentence[i - 1][1]
           transition_counts[prev_tag][tag] += 1

tags = sorted(tag_counts.keys())
```

```
vocab = sorted({word.lower() for word, _ in
tagged_sentence})

tag_index = {tag: i for i, tag in enumerate(tags)}
word_index = {word: i for i, word in enumerate(vocab)}

N = len(tags)
V = len(vocab)

start_prob = np.zeros(N)
transition_matrix = np.zeros((N, N))
emission_matrix = np.zeros((N, V))

for tag in tags:
   start_prob[tag_index[tag]] = start_counts[tag]

start_prob = start_prob / start_prob.sum()

for prev_tag in tags:
   for tag in tags:

transition_matrix[tag_index[prev_tag]][tag_index[tag]] =
transition_counts[prev_tag][tag]

if transition_matrix.sum(axis=1).any():
   transition_matrix = np.divide(
       transition_matrix,
       transition_matrix.sum(axis=1, keepdims=True),
       where=transition_matrix.sum(axis=1, keepdims=True)
!= 0
   )

# Emission matrix
for tag in tags:
   for word in vocab:
       emission_matrix[tag_index[tag]][word_index[word]] =
emission_counts[tag][word]

if emission_matrix.sum(axis=1).any():
   emission_matrix = np.divide(
       emission_matrix,
       emission_matrix.sum(axis=1, keepdims=True),
```

```
        where=emission_matrix.sum(axis=1, keepdims=True) !=
0
    )

print("\nFull Transition Matrix:\n")
print(transition_matrix)

print("\nFull Emission Matrix:\n")
print(emission_matrix)

sentence = [word.lower() for word, tag in tagged_sentence]

def get_possible_tags(word):
    return [tag for tag in tags if
emission_counts[tag][word] > 0]

def manual_probability(sentence, tag_sequence):
    prob = start_prob[tag_index[tag_sequence[0]]] * \

emission_matrix[tag_index[tag_sequence[0]]][word_index[sen
tence[0]]]

    for i in range(1, len(sentence)):
        prob *= transition_matrix[tag_index[tag_sequence[i
- 1]]][tag_index[tag_sequence[i]]]
        prob *=
emission_matrix[tag_index[tag_sequence[i]]][word_index[sen
tence[i]]]

    return prob

possible_tags_per_word = [get_possible_tags(word) for word
in sentence]
all_paths = list(product(*possible_tags_per_word))

best_path = None
best_prob = 0

for path in all_paths:
    prob = manual_probability(sentence, path)
    if prob > best_prob:
        best_prob = prob
```

```
        best_path = path

model = hmm.CategoricalHMM(n_components=N, init_params="")
model.startprob_ = start_prob
model.transmat_ = transition_matrix
model.emissionprob_ = emission_matrix

obs = np.array([[word_index[w]] for w in sentence])

log_prob_score = model.score(obs)
library_prob = np.exp(log_prob_score)

log_prob_decode, state_sequence = model.decode(obs)
library_path = [tags[state] for state in state_sequence]

print("\nManual Best Path:", best_path)
print("Manual Probability:", best_prob)

print("\nLibrary Best Path:", library_path)
print("Library Probability:", library_prob)
```

RESULT:

```
NLP on  main [!] is  v0.1.0 via  v3.10.13 (NLP)
❯ uv run Exp4/exp4.py
[nltk_data] Downloading package brown to /home/raj_99/nltk_data...
[nltk_data]   Package brown is already up-to-date!
[nltk_data] Downloading package universal_tagset to
[nltk_data]     /home/raj_99/nltk_data...
[nltk_data]   Package universal_tagset is already up-to-date!

Sentence Used For Computation:

['daniel', 'personally', 'likes', 'to', 'read']

Tagged Version:

[('Daniel', 'NOUN'), ('personally', 'ADV'), ('likes', 'VERB'), ('to', 'PRT'), ('read', 'VERB')]

Full Transition Matrix:

[[0. 0. 0. 1.]
 [1. 0. 0. 0.]
 [0. 0. 0. 1.]
 [0. 0. 1. 0.]]

Full Emission Matrix:

[[0.  0.  1.  0.  0. ]
 [1.  0.  0.  0.  0. ]
 [0.  0.  0.  0.  1. ]
 [0.  0.5 0.  0.5 0. ]]

Manual Best Path: ('NOUN', 'ADV', 'VERB', 'PRT', 'VERB')
Manual Probability: 0.25

Library Best Path: ['NOUN', 'ADV', 'VERB', 'PRT', 'VERB']
Library Probability: 0.25
```

CONCLUSION:

In this experiment, we implemented a **Hidden Markov Model (HMM)** for Part-of-Speech (POS) tagging using the Brown Corpus with the universal tagset. We computed the **start probabilities, transition probabilities, and emission probabilities** from frequency counts of tags and words. Using these probabilities, we manually calculated the most probable tag sequence for a selected sentence and compared it with the sequence predicted by the HMM library implementation.

The results showed that the manually computed best path matched the library's decoded path, confirming the correctness of the probability calculations and model construction. This experiment demonstrates how statistical frequency-based learning forms the foundation of probabilistic sequence models and highlights how HMM effectively models sequential dependencies in natural language.

Overall, the experiment successfully validates the working of Hidden Markov Models for POS tagging.